

PROJECT REPORT
ON
Expense Splitter — Command Line Application

Submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By : Stuti Jain (25BAI10327)



**SCHOOL OF COMPUTING SCIENCE AND ENGINEERING AND ARTIFICIAL
INTELLIGENCE**

VIT BHOPAL UNIVERSITY

KOTHRIKALAN, SEHORE

MADHYA PRADESH – 466114

SEP 2026

INTRODUCTION

Anyone who has been away on a vacation with friends or lived in a shared apartment with roommates knows how challenging it could be to track who spent what and with whom. Some people pay for dinners, some split the cost of a hired taxi in cash, others pick up groceries for the week and so on – and by the end of the trip, no one seems to remember the details. This project is dedicated to solving a problem of this kind.

I have created a command line Java application that allows a social circle of people to track their expenses and determine who owes what to whom in order to equalize the settlements. The program helps minimize the number of compensation operations required to balance all debts within a group of people. Instead of making every member pay every other member that they indebted, the app calculates the optimal set of transactions that settles all balances with the least number of operations.

In addition to the obvious utility of such a program, the assignment also served to demonstrate the practical application of the object-oriented programming principles. I took great care to implement all the OOP features we have learned in the course. Specifically, the application makes extensive use of encapsulation and abstraction and serves as an example of polymorphism and efficient exception handling. While developing the program, I tried, as far as possible, to adhere to the basic principles of OOP design, and I will show in the following sections how each of these principles was applied when developing the application.

The software is completely command line-based and is designed to keep things as simple as possible for the user. It stores the data only for the current session because there is no need for a database or other persistent storage. The main focus is on performing the required operations for the current session as efficiently as possible.

PROBLEM STATEMENT

Manually dividing the costs among a social circle of friends is not as simple as it seems. Defining an equal share for every participant based on the expenses made seems logical, but it becomes a complex mathematical task when the number of friends and irregularities in the distribution of cash increases. Moreover, there is also a problem of inefficient division that arises even if the balances were initially calculated correctly.

When calculating the settlements, most people consider only the first-order transactions, that is, the debts that arise directly from the fact of the expenditure. However, when there are multiple expenses in the system, the number of transactions required to equalize all the debts is much higher than the number of participants. For example, Rahul may have to pay 200 Rupees to Stuti for one of the dinners. At the same time, Stuti owes Rahul 150 Rupees for the previous taxi fare. The most obvious way of equalizing the debts would be for Rahul to receive 150 Rupees from Stuti and, in turn, he would pay 200 Rupees to her. However, in reality, a single transaction of 50 Rupees from Rahul to Stuti would be sufficient to equalize all the balances.

This application was created in order to address exactly these kinds of problems. It defines a set of operations that equalize all the debts within a social circle with a minimal number of transactions. The application therefore has a built-in system for defining the first-order transactions with the ability to set equal, proportional, or individual shares for each participant. The program analyzes the system of debts and determines the simplest structure of settlements. The user gets a concise representation of who owes what to whom at the end of all the operations. By using this application, users can customize every transaction detail to fit their needs.

FUNCTIONAL REQUIREMENTS

The application shall have the following functionality:

1. Member management: Adding a new member to the current group with a unique ID and a display name. This feature is necessary since every expense has to be associated with an owner and every debt has to be related to specific members.

2. Adding an expense: A new expense can be added with a description, the overall amount, a payer who covers the expense, and a set of members that share this expense with the payer. This function is the main one and is used to input the data required to calculate the debts.

3. Define split type: Every expense has to be split into shares with an option to set equal payoffs, set percentages and exact amounts for each member. This function adds flexibility since there are cases when members have to cover different portions of the same expense.

4. View balances: The current balances can be reviewed at any time to see who is indebted and who is owed money.

5. Calculate settlements: The user can calculate the minimal set of transactions required to equalize all the debts.

6. Export current balances: The current balances and the settlements can be exported into a text file to review the balances outside of the application since the dataset is not persistent after a session ends.

These functions allow the user to operate with a high degree of flexibility. Every expense in the social circle can be reviewed in detail, and every aspect of the payoff can be customized. Using these functions in a particular order allows the user to obtain a balanced set of transactions from the application.

NON-FUNCTIONAL REQUIREMENTS

1. Usability — The interface is entirely menu-driven and uses numbered options, so that a user facing the start-up screen for the first time does not need to read documentation to understand how to proceed — every prompt informs the user exactly what input is expected.

2. Reliability — The program is resistant to incorrect input. Entering letters instead of digits, referencing a member that doesn't exist, or providing percentages that don't add up to 100%, for instance, will trigger an informative error message instead of a generic unhandled exception and consequent termination.

3. Maintainability — The code is organized in a way that makes it easy for maintainers (including me) to find, read, and understand individual sections — model, strategy, exception, and engine are separated as logical packages, and new features (like a new type of split ratio) can be added without having to consider or modify unrelated components.

4. Error Handling Strategy — Instead of using Java's generic exceptions, this project utilizes two custom checked exceptions to better communicate the exact cause of an error to the user. `InvalidSplitException` is thrown whenever an expense's split ratios fail validation, and `MemberNotFoundException` is thrown when a referenced member doesn't exist.

5. Resource Efficiency — As a CLI utility designed for a single session, this project does not have a significant need for optimization, but unnecessary objects are not created, and calculations that can be deferred (such as balance recalculation after every menu option) are postponed for when they are actually needed.

SYSTEM ARCHITECTURE

The application is designed in layers, to separate concerns and promote code reuse:

Presentation Layer — Main.java exclusively deals with presentation logic: menu display, user input (handled by Scanner), and routing the input to the relevant methods. This layer is decoupled from the actual calculation logic, to avoid conflation between domain model and presentation.

Domain Model Layer — the model package contains classes that represent real-life entities (Member, Expense, Group). These focus on holding state and exposing it for consumption by other layers.

Strategy Layer — the strategy package contains the SplitStrategy interface and its(EqualSplit, PercentageSplit, ExactAmountSplit) implementations. This layer is concerned exclusively with defining and implementing methods to split an amount between members, without regard for the domain model or actual calculations.

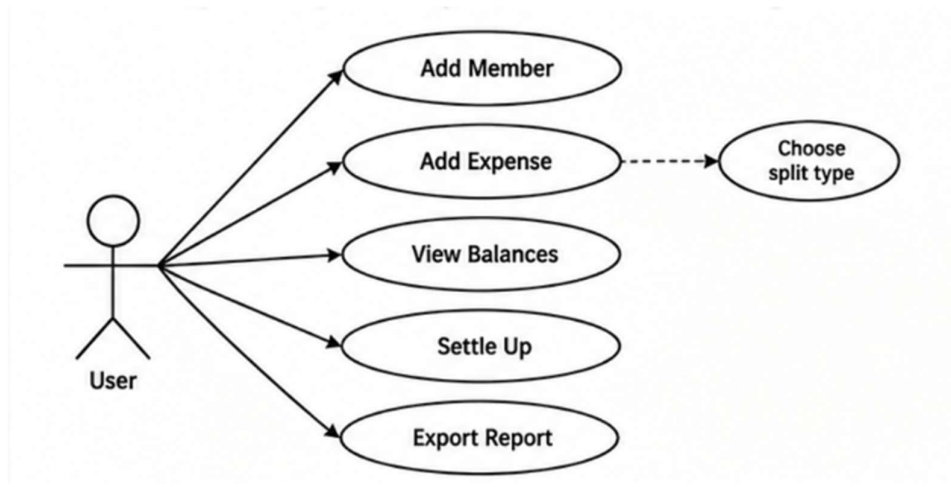
Business Logic Layer — the engine package contains the SettlementEngine class, which implements the logic to calculate net balances for all members and simplify the debts between them. It is intentionally agnostic to the method of calculation — any algorithm that provides a Group and a valid split strategy will work.

Exception Layer — the exception package contains two custom checked exceptions used in this project: InvalidSplitException and MemberNotFoundException.

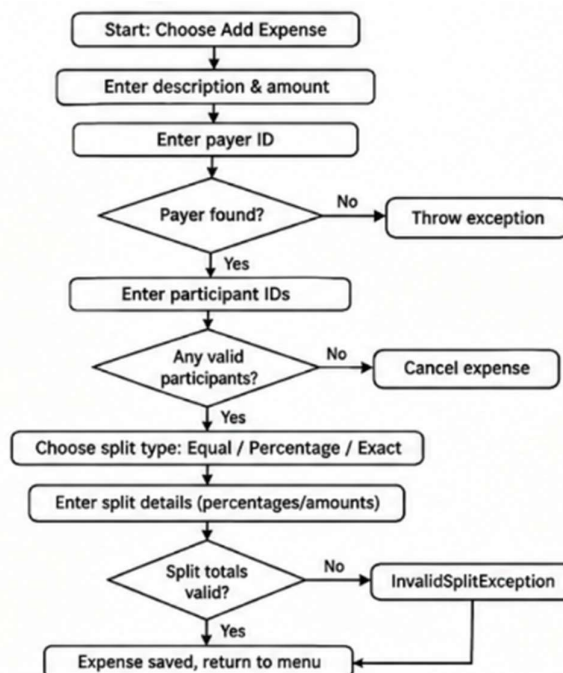
This separation allows me to, for instance, introduce a database later by modifying only the presentation tier: existing code will continue to work with whatever Group implementation is provided to it, strategy or otherwise.

DESIGN DIAGRAMS

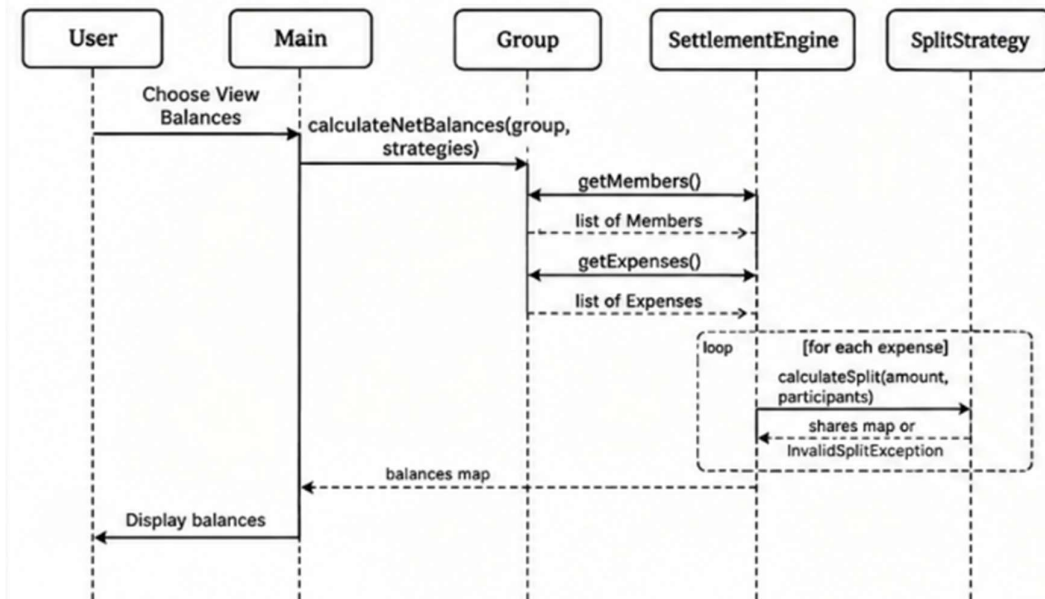
1. Use Case Diagram: — Shows the single User actor and the six primary interactions with the system: Add Member, Add Expense, View Balances, Settle Up, and Export Report, with Choose Split Type modeled as a sub-step included within Add Expense.



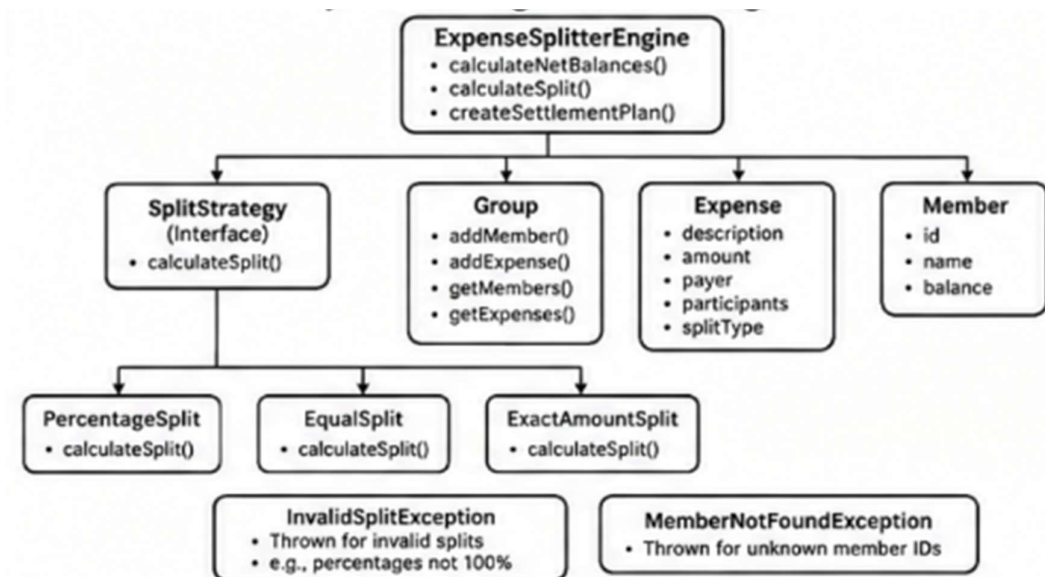
2. Workflow Diagram: — Walks through exactly what happens, step by step, when a user adds an expense — including the points where the system checks for a valid payer, valid participants, and a mathematically valid split, and what happens on each of those checks failing.



3. Sequence Diagram: — Shows the order of method calls between Main, Group, SettlementEngine, and SplitStrategy during a "View Balances" request, including the loop where each expense's split is recalculated.



4. Class/Component Diagram: — Lays out the full class structure of the project: all model classes, the SplitStrategy interface and its three implementations, SettlementEngine, and both custom exceptions, along with how they all relate to each other (composition, dependency, interface implementation).



DESIGN DECISIONS AND RATIONALE

Why the Strategy pattern for splits: My first idea was to implement a single method with conditional logic that would perform different calculations depending on the split type. However, this would involve significant rewriting of this method every time a new split type is added, and conditional logic poses a maintenance liability in general. By using the Strategy pattern, each split type is self-contained, and SettlementEngine does not need to know or care which particular split strategy it is using. This is also the rare opportunity in this project to demonstrate polymorphism.

Why custom exceptions: while it would be possible to throw generic unchecked exceptions such as `IllegalArgumentException`, using checked exceptions allows me to clearly communicate exceptional circumstances to the caller while providing relevant implementation details (such as the member reference that caused a `MemberNotFoundException`). Custom exceptions also let me differentiate between various error cases, such as an invalid split ratio or a non-existent member.

Why a greedy algorithm for settling debts: it would be possible to simply list all debts arising from expenses, but the more interesting option is to minimize the number of transactions by always paying off the largest debts first. This approach is suboptimal for some edge cases (and there are more sophisticated algorithms to handle them), but it is simple to implement and, for a small group of people, will always settle debts in the fewest possible transactions. It also demonstrates an understanding of greedy algorithms.

Why no persistent storage: I deliberately omitted it, because the time spent on it would be better invested elsewhere: this is a learning project, and its purpose is to practice OOP concepts, not to design a file format or learn a database system. It is explicitly mentioned as a limitation.

IMPLEMENTATION DETAILS

The project is written in pure Java (JDK 21, but JDK 8 should suffice) and does not require any external dependencies.

Member, Expense, and Group are standard POJOs with private fields and public getters, and Member has a special `adjustBalance()` method instead of a public setter. The split ratio calculation methods are implemented in the strategy package, with `EqualSplit` having no validation logic (it always throws an `InvalidSplitException` if the split ratio is not 100%), `PercentageSplit` validating that all percentages add up to approximately 100%, and `ExactAmountSplit` ensuring that all specified amounts add up to the total expense. Any invalid input results in an `InvalidSplitException` being thrown with an explanatory message.

`SettlementEngine`'s `calculateNetBalances()` method iterates through all expenses in a group, using whatever `SplitStrategy` was specified for that expense (stored in a map in the Main class), and accumulates the calculated debts for each member. `simplifyDebts()` then acts on these net balances: it iteratively identifies the members that are owed the most money and those that owe the most, offsetting their debts until all balances are settled.

The Main class is largely concerned with presentation logic: it displays the menu and processes the input with `Scanner`, delegating actual calculations to `SettlementEngine`. The interaction loop continues until the user selects the "Exit" option. A helper method `readValidDouble()` is used to ensure that the input parser (`Scanner`) does not throw an exception when given invalid input; instead, the user is prompted to enter a number again.

RESULTS

➤ Menu display

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Stuti Jain\OneDrive\Desktop\expense-splitter> & 'C:\Program Files\Java\jdk-21.0.12\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
' 'C:\Users\Stuti Jain\AppData\Roaming\Code\User\workspaceStorage\99870ae3fe77593ced10b5a919466250\redhat.java\jdt_ws\expense-splitter_be71f87d\bin' 'com.sp
litter.Main'
Enter your group name: Goa Trip

===== Expense Splitter (Goa Trip) =====
1. Add Member
2. Add Expense
3. View Balances
4. Settle Up
5. Export Settlement Report
6. Exit
Choose an option: |
```

➤ Add Member

```
Problems Output Debug Console Terminal Ports

===== Expense Splitter (Goa Trip) =====
1. Add Member
2. Add Expense
3. View Balances
4. Settle Up
5. Export Settlement Report
6. Exit
Choose an option: 1
Enter member ID: m1
Enter member name: Stuti
Stuti added successfully.
```

```
Problems Output Debug Console Terminal Ports

===== Expense Splitter (Goa Trip) =====
1. Add Member
2. Add Expense
3. View Balances
4. Settle Up
5. Export Settlement Report
6. Exit
Choose an option: 1
Enter member ID: m2
Enter member name: Rahul
Rahul added successfully.
```

➤ Add Expense

```
===== Expense Splitter (Goa Trip) =====
1. Add Member
2. Add Expense
3. View Balances
4. Settle Up
5. Export Settlement Report
6. Exit
Choose an option: 2
Enter expense description: Dinner
Enter total amount: 1000
Enter payer's member ID: m1
Enter participant IDs separated by commas (e.g. m1,m2,m3): m1,m2
Choose split type: 1. Equal  2. Percentage  3. Exact Amount
2
Enter percentage for Stuti: 40
Enter percentage for Rahul: 60
Expense added successfully.
```

➤ View Balances

```
Problems  Output  Debug Console  Terminal  Ports

===== Expense Splitter (Goa Trip) =====
1. Add Member
2. Add Expense
3. View Balances
4. Settle Up
5. Export Settlement Report
6. Exit
Choose an option: 3

--- Current Balances ---
Rahul owes Rs.600.00
Stuti is owed Rs.600.00
```

➤ Settle Up

```
Problems  Output  Debug Console  Terminal  Ports

===== Expense Splitter (Goa Trip) =====
1. Add Member
2. Add Expense
3. View Balances
4. Settle Up
5. Export Settlement Report
6. Exit
Choose an option: 4

--- Settlement Plan ---
Rahul pays Stuti Rs.600.00
```

➤ Export Settlement Report

```
===== Expense Splitter (Goa Trip) =====  
1. Add Member  
2. Add Expense  
3. View Balances  
4. Settle Up  
5. Export Settlement Report  
6. Exit  
Choose an option: 5  
Report exported successfully to settlement_report.txt
```

➤ Exit

```
===== Expense Splitter (Goa Trip) =====  
1. Add Member  
2. Add Expense  
3. View Balances  
4. Settle Up  
5. Export Settlement Report  
6. Exit  
Choose an option: 6  
Goodbye!  
PS C:\Users\Stuti Jain\OneDrive\Desktop\expense-splitter>
```

TESTING APPROACH

I did not write any automated tests for this project — testing was done manually by interacting with the running program and observing the results.

I tested that adding multiple members works by subsequently listing them in the output. I tested that calculations are performed correctly for all split types by manually entering test data and verifying the results. Attempting to enter percentages that do not add up to 100% (e. g. 40 and 40) resulted in an `InvalidSplitException` being thrown, as did incorrect exact amounts for an expense. I also tested that providing invalid member references as payers or participants causes a `MemberNotFoundException` to be thrown, and that attempting to create an expense with no valid participants cancels the operation (to avoid a divide-by-zero exception in `EqualSplit`). Entering letters where numbers are expected resulted in the prompt being repeated, as the input could not be parsed as a double. Finally, I verified that the algorithm correctly minimizes the number of transactions by demonstrating that it does not simply list every debt, but instead offsets them where possible.

Each test was performed by launching `java com.splitter.Main` and interacting with the program in the manner described.

CHALLENGES FACED

The greatest challenges in this project were not directly related to implementing the OOP concepts, but rather came from the development environment, which provided excellent learning material on its own.

A common issue was that files I edited would become empty (reportedly 0 bytes) after I attempted to write to them, resulting in confusing compiler errors that indicated missing classes or methods when, in reality, the file was empty. This would happen both when I tried to edit the file in an IDE or even in a text editor. I learned to be especially wary of such issues after seeing that `dir` reported the file as 0 bytes — in practice, it often helped to simply try writing the file again.

I had similar issues with Git, specifically with the `.gitignore` file: despite my best efforts, it never seemed to work, and Git would track all files, including `.class` and reports directories. In the end, I learned that the issue was caused by PowerShell's `echo >` redirection using UTF-16 encoding by default, whereas Git expects UTF-8. This can be fixed by using `Set-Content -Encoding utf8` instead.

On the programming side, I encountered the infamous Scanner issue where the call to `nextInt()` left a newline character in the input buffer, which would then be erroneously read as the next line of input. I had never actually seen this issue before, and it took some time to troubleshoot.

Finally, adding exception handling to the split strategy methods after initially writing them without required updating numerous files, including the method signature in four places and all calls to `calculateSplit()`. It was an eye-opening experience that reinforced the importance of designing method contracts (especially interfaces) with future maintainers in mind.

LEARNINGS AND KEY TAKEAWAYS

Working on this project gave me a significantly better practical understanding of the Strategy design pattern, as I implemented it from design and saw the benefits of using it in the code. In particular, adding new split strategies became a matter of writing a single class implementing the existing interface, without having to make any changes to unrelated code.

I also gained a better appreciation for the strengths and weaknesses of checked exceptions. In particular, making sure that `PercentageSplit` and `ExactAmountSplit` could not fail in unforeseen ways forced me to make sure that all possible error conditions were properly handled. At the same time, declaring `InvalidSplitException` as a checked exception meant that any code using these classes (or indeed, any code in this project) had to handle it explicitly, which in practice mostly meant wrapping it in a `RuntimeException` with a descriptive message.

Practically, I gained experience not only with Git, but with the command-line interface in general, and feel much more confident about using it. I also learned first-hand that most of programming is not designing new algorithms or data structures, but rather, working within existing ones to perform the required operations and handle edge cases.

FUTURE ENHANCEMENTS

If I were to continue working on this project, the next logical steps would be to implement persistent storage (file or database-backed) and to support multiple currencies (as opposed to using a hardcoded Rupee symbol). Adding proper automated tests (JUnit) would also be a valuable exercise, particularly for the SettlementEngine class, as it performs particularly complex calculations that could benefit from verification.

A more involved enhancement would be the implementation of a "shares"-based split ratio, where members are assigned relative weights (e. g. "2" and "1") rather than a percentage or an exact amount.

Finally, it would be comparatively simple to implement a basic web or GUI frontend for this application, as the presentation logic is separated from the rest of the code.

REFERENCES

- Oracle Java Documentation — <https://docs.oracle.com/en/java/>
- Gamma, E., Helm, R., Johnson, R., Vlissides, J. — *Design Patterns: Elements of Reusable Object-Oriented Software* (referenced for the Strategy pattern)
- Java Collections Framework Documentation — <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>
- Oracle's official Java Exceptions tutorial — <https://docs.oracle.com/javase/tutorial/essential/exceptions/>